305432 เครือข่ายเซนเซอร์ (Sensor Networks)

Publish-Time Optimizations for Web-Based Visualizations

การเพิ่มประสิทธิภาพเว็บแสดงภาพข้อมูลเชิงโต้ตอบด้วยการประมวลผลล่วงหน้าช่วงเผยแพร่

Ron Pechuk, Jeffrey Heer
University of Washington

2025 IEEE Visualization and Visual Analytics (VIS), pp. 226–230

IEEE Doc ID: 11298742 DOI: 10.1109/VIS60296.2025.00051

นำเสนอโดย นายธีรภัทร ภู่ระย้า รหัสนิสิต 66362416

ที่มา ปัญหาความหน่วง และตัวชี้วัด

ปัญหาเมื่อเปิดเว็บแสดงภาพข้อมูล

1 แต่ละเซสชันต้องคำนวณข้อมูลใหม่

เมื่อเปิดเว็บ เบราว์เซอร์ต้องโหลดข้อมูล รันคิวรี และวาดภาพ
การทำงานซ้ำนี้เพิ่มเวลารอ โดยเฉพาะกับข้อมูลขนาดใหญ่

2 แคชยังไม่พร้อมในช่วงเริ่มเซสชัน

แคชที่สร้างระหว่างใช้งานยังไม่มีผลให้ใช้ตอนเริ่มต้น
จึงต้องรอคำนวณก่อน แม้คำขอถัดไปจะใช้แคชได้ (Cold Start)

3 ภาพแสดงแล้ว แต่ระบบยังรับอินพุตไม่ได้

การกรองหรือซูมต้องรอคอมโพเนนต์และตัวกลางคิวรีเริ่มทำงาน
การเห็นภาพจึงไม่เท่ากับระบบพร้อมรับการโต้ตอบ

ตัวชี้วัดเวลาและพื้นที่จัดเก็บ

1 TTR: เวลาจนภาพข้อมูลปรากฏครบ

Time-to-Render วัดตั้งแต่ส่งคำขอจนภาพข้อมูลครบถ้วน
ต่างจาก FCP ซึ่งนับเนื้อหาแรกที่ปรากฏบนหน้าเว็บ

2 TTA: เวลาจนระบบพร้อมโต้ตอบ

Time-to-Activation วัดจนพร้อมรับการเลือก กรอง หรือซูม
เทียบแนวคิดกับ TTI แต่ไม่ใช่เวลาประมวลผลทุกการคลิก

3 SC: ขนาดทรัพยากรทั้งหมดที่เผยแพร่

Storage Cost รวมข้อมูล สคริปต์ ภาพ และผลคำนวณล่วงหน้า
ใช้เทียบกับกรณีไม่ปรับปรุง (none) เพื่อประเมินต้นทุนพื้นที่

ที่มา: Sections 1 และ 3.1 (หน้า PDF 1–2)

สถาปัตยกรรม Mosaic Publisher

งานสามกลุ่มที่ย้ายไปทำก่อนเปิดเว็บ

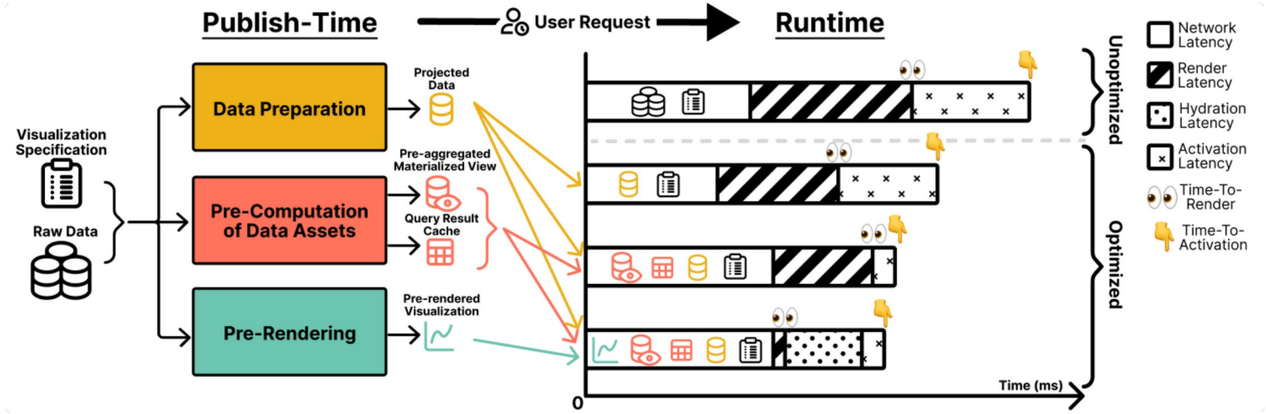


Figure 1: เตรียมข้อมูล ผลคิวรี และภาพก่อนคำขอ แล้วใช้ผลในช่วงรันไทม์ (PDF หน้า 1)

ขั้นตอนเตรียมทรัพยากรของ Publisher

1. อ่านข้อกำหนดและจำลองอินพุต → **2. ใช้ DuckDB คำนวณและบันทึกผลล่วงหน้า** → **3. แก้ข้อกำหนดและสร้าง JavaScript**

วิเคราะห์โครงสร้างข้อกำหนด (AST) และจำลองอินพุตใน Virtual DOM (PDF หน้า 3)

ทรัพยากรที่ส่งไปยังเบราว์เซอร์

1 ข้อมูลที่เตรียมพร้อมใช้

เลือกคอลัมน์และแปลงข้อมูลก่อนเผยแพร่ เบราว์เซอร์จึงโหลดข้อมูลที่พร้อมวิเคราะห์

2 ผลคิวรีและผลรวมล่วงหน้า

เติมแคชและฐานข้อมูล DuckDB-WASM Coordinator ใช้ผลที่เตรียมแทนคำนวณซ้ำ

3 ภาพเริ่มต้นและสคริปต์โต้ตอบ

ส่งภาพที่วาดแล้วให้แสดงก่อน พร้อม JavaScript เปิดการโต้ตอบภายหลัง

ที่มา: Sections 3.2–3.4 และ 4.1–4.3 (หน้า PDF 2–3)

ภาพล่วงหน้าและการเริ่มระบบโต้ตอบ

เวลาที่ใช้โหลดภาพและเริ่มระบบโต้ตอบ

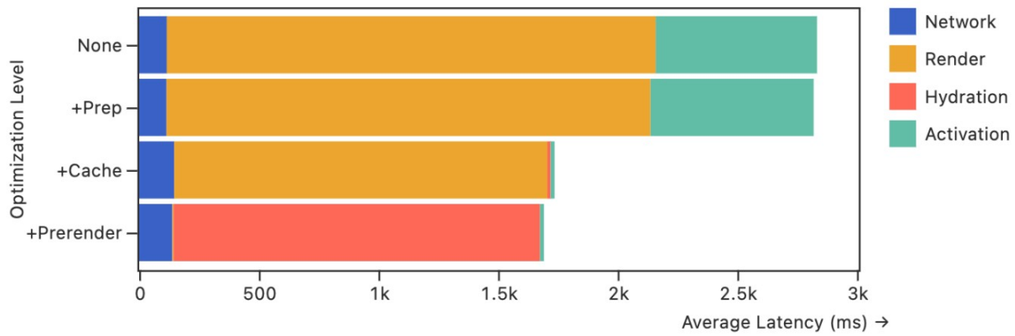


Figure 4: เวลาเฉลี่ยแยกตามขั้นตอน จาก 5 รูปแบบภาพที่ข้อมูล 10 ล้านแถว (PDF หน้า 4)

เห็นภาพก่อน แล้วจึงเปิดการโต้ตอบ

1. ภาพปรากฏครบ ถึงจุดวัด TTR → **2. เริ่มคอมโพเนนต์ ฝั่งไคลเอนต์ (Hydration)** → **3. พร้อมกรองและซูม ถึงจุดวัด TTA**

ผังแสดงขั้นตอนตาม Section 4.3 ไม่ได้แสดงสัดส่วนเวลา (PDF หน้า 3)

ภาพสองประเภทและต้นทุน Hydration

1 ภาพสถิต (Static Views)

ส่งเนื้อหาที่วาดไว้โดยไม่ต้องเพิ่ม สคริปต์สำหรับการโต้ตอบ

2 ภาพโต้ตอบ (Interactive Views)

ส่งภาพพร้อม JavaScript เพื่อเริ่มคอมโพเนนต์ และส่วนรับอินพุต ก่อนกรองหรือซูมได้

3 Hydration ยังเป็นคอขวด

วิธีที่ทดลองใช้ต้นทุนใกล้การเรนเดอร์เต็มชุด ผู้วิจัยเสนอให้ลดงานเริ่มคอมโพเนนต์ต่อไป

ที่มา: Sections 4.3 และ 5 (หน้า PDF 3–4)

ผลการทดลอง: เวลาเปิดภาพและต้นทุนพื้นที่

ผล TTR และ TTA เมื่อขนาดข้อมูลเพิ่มขึ้น

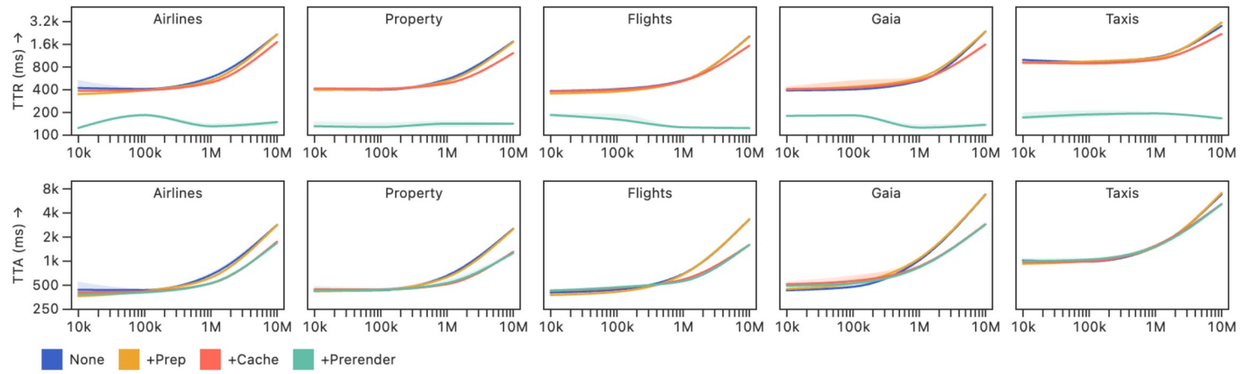


Figure 3: เส้นมัธยฐาน (Median) และแรเงา IQR จากข้อมูล 10k–10M แถว (หน้า PDF 4)

ผลเทียบกรณีไม่ปรับปรุง (none)

ระดับ	ΔSC %	ΔTTR %	ΔTTA %
+prep	0.0 ± 0.0	−0.6 ± 4.0	−1.0 ± 2.5
+cache	21.1 ± 29.8	−16.6 ± 1.3	−32.3 ± 8.9
+most	21.2 ± 29.9	−83.7 ± 2.1	−33.3 ± 9.6

Table 1: ค่าเฉลี่ย ± SD ของการเปลี่ยนแปลง (%)

+most คือระดับรวมที่เพิ่มภาพล่วงหน้า
ในกราฟใช้ชื่อ +prerender

−83.7%

เวลาจนภาพข้อมูลปรากฏครบ (TTR)

ชุดรวมเทียบ none · SD = 2.1%

−33.3%

เวลาจนระบบพร้อมโต้ตอบ (TTA)

แคชลด TTA 32.3% · ชุดรวม SD = 9.6%

+21.2%

ขนาดทรัพยากรที่เผยแพร่ (SC)

พื้นที่เพิ่มเฉลี่ย · SD = 29.9%

สภาพแวดล้อม: MacBook Pro (M1 Pro, RAM 16 GB), Node 22.9.0, Local Server, DuckDB-WASM

TTA ลดเพิ่มจาก +cache อีก 1.0 จุดเปอร์เซ็นต์ (33.3 – 32.3) เมื่อเทียบฐาน none เดียวกัน

ที่มา: Figure 3 และ Table 1 (PDF หน้า 4) · ตารางเฉลี่ยข้าม 5 รูปแบบภาพและข้อมูล 10k–10M แถว

ผลงานวิจัยและข้อจำกัดของการทดลอง

ผลงานสำคัญของบทความ

1 พัฒนา Publisher บนระบบ Mosaic

เตรียมข้อมูล ผลคิวรี และภาพเริ่มต้นจากข้อกำหนดเดิม
เติมแคชผ่านตัวกลางคิวรี และแก้แหล่งข้อมูลเฉพาะส่วนที่จำเป็น

2 ประเมินทั้งการแสดงภาพและการโต้ตอบ

ใช้ TTR และ TTA แยกเวลารอของสองขั้นตอน
พร้อมวัด SC เพื่อรายงานต้นทุนพื้นที่ของการปรับปรุง

3 เปรียบเทียบการปรับปรุงทีละระดับ

ทดสอบ 5 รูปแบบภาพและข้อมูล 10k–10M แถว
พบว่าแคชช่วย TTA ส่วนภาพลวงหน้าช่วย TTR เด่นกว่า

ข้อจำกัดของการทดลอง

1 ยังไม่ครอบคลุมอุปกรณ์และเครือข่ายอื่น

ทดสอบบน M1 Pro กับ local server
ยังไม่มีผลบนมือถือหรือเครือข่ายระยะไกล

2 ยังมีต้นทุนเริ่มระบบและพื้นที่เพิ่ม

Hydration ใช้วิธีสร้างคอมโพเนนต์ตามปกติ
SC เพิ่มเฉลี่ย 21.2% และต่างกันมากระหว่างกรณี (SD 29.9%)

3 การอัปเดตข้อมูลและรายละเอียดการวัด

ข้อมูลที่เปลี่ยนต้องสร้างทรัพยากรใหม่
เอกสารไม่ได้ระบุจำนวนรอบทดลองซ้ำ

ที่มา: Sections 3–5 (PDF หน้า 2–4) · พิจารณาข้อจำกัดตามสภาพแวดล้อมและข้อมูลที่รายงาน

สรุปผลและแนวทางวิจัยในอนาคต

ผลที่ได้จากการทดลอง

1 แคชล่วงหน้าลดเวลาจนพร้อมโต้ตอบ

การเพิ่มแคชและผลรวม (+cache) ลด TTA เฉลี่ย 32.3%
ช่วยลดงานคิวรีที่ต้องคำนวณตอนเปิดเว็บ

2 ภาพล่วงหน้าทำให้เห็นข้อมูลเร็วขึ้น

ชุดรวมลด TTR เฉลี่ย 83.7% และ TTA 33.3%
ภาพแสดงเร็วขึ้น แต่ยังต้องรอเริ่มคอมโพเนนต์โต้ตอบ

3 การลดเวลารอต้องแลกกับพื้นที่จัดเก็บ

SC เพิ่มเฉลี่ย 21.2% (SD 29.9%) เทียบ none
ต้นทุนพื้นที่ต่างกันตามรูปแบบภาพและขนาดข้อมูล

แนวทางที่ผู้วิจัยเสนอ

1 ลดภาระการเริ่มคอมโพเนนต์

ศึกษาวิธี Hydration ที่ละเอียดขึ้น
เพื่อลดต้นทุนที่ยังใกล้การเรนเดอร์เต็มชุด (§5)

2 ผสานการคำนวณและสร้างใหม่เฉพาะส่วน

ใช้ทั้งช่วงเผยแพร่และรันไทม์ (Hybrid) และสร้างใหม่
เฉพาะส่วนที่เปลี่ยน (Incremental Regeneration, §6)

3 เลือกระดับการปรับปรุงโดยอัตโนมัติ

พัฒนาวิธีเลือกการตั้งค่าจากข้อมูลและข้อกำหนดภาพ
ให้เหมาะกับประโยชน์และต้นทุนของแต่ละกรณี (§6)

ที่มา: ผลทดลอง Table 1 / Figure 4 · แนวทางอนาคต Sections 5–6 (PDF หน้า 4) ยังไม่ได้ทดสอบ